

# UTAMACS Mobile App — Complete Self-Sufficient Runbook

---

**App name:** UTA MACS Resident Portal

**Package ID:** `org.utamacs.utamacs_portal`

**Platform:** Flutter (Dart) — iOS + Android

**Author:** Subramanyam Theerthala

**Last updated:** May 2026

**Purpose of this document:** You should be able to rebuild, run, test, debug, and ship this app to the App Store and Play Store using only this document — no external help needed. Every command is written out in full. Every concept is explained from first principles. If you are new to Flutter and Dart, start at §A. If you are resuming work, jump to the section you need.

---

## Table of Contents

### Part I — Background & Decisions

- [§1 How we got here — the decision journey](#)
- [§2 Why Flutter + Dart — the full reasoning](#)

### Part II — Flutter & Dart for the Newcomer

- [§3 Dart language primer — everything you need to know](#)
- [§4 Flutter fundamentals — how the framework works](#)
- [§5 Key packages used in this project — why and how](#)

### Part III — Setting Up from Zero

- [§6 Machine setup — install everything](#)
- [§7 Project creation and initial scaffold](#)
- [§8 Environment configuration \(.env\)](#)

### Part IV — Codebase Deep-Dive

- [§9 Repository structure](#)
- [§10 Architecture — how the app is built](#)
- [§11 Design system — tokens mapped from the web portal](#)
- [§12 Authentication flow — step by step](#)
- [§13 State management with Riverpod](#)
- [§14 Supabase integration](#)
- [§15 Navigation model \(go\\_router\)](#)
- [§16 How Astro portal pages were ported to Flutter](#)
- [§17 All 28 modules — what exists today](#)
- [§18 Shared components](#)

### Part V — Running & Testing Locally

- [§19 Running the app — every platform](#)
- [§20 Using the iOS Simulator](#)
- [§21 Using the Android Emulator](#)
- [§22 Testing on a physical device](#)
- [§23 Hot reload, hot restart, and debugging](#)
- [§24 Writing and running tests](#)
- [§25 Common errors and how to fix them](#)

## Part VI — Building Features

- [§26 Adding a new screen \(step by step\)](#)
- [§27 Adding a new Supabase query](#)
- [§28 Adding a form with validation](#)
- [§29 Adding file/image upload from mobile](#)
- [§30 Push notifications](#)

## Part VII — Deploying to Production

- [§31 Platform preparation — Android signing](#)
- [§32 Platform preparation — iOS signing and certificates](#)
- [§33 Building release binaries](#)
- [§34 Deploying to Google Play Store](#)
- [§35 Deploying to Apple App Store](#)
- [§36 CI/CD pipeline with GitHub Actions](#)

## Part VIII — Maintenance

- [§37 Upgrading packages](#)
- [§38 Commit history — what landed and when](#)
- [§39 Known issues and bugs fixed](#)
- [§40 Next steps and roadmap](#)
- [§41 Reference links](#)

---

# PART I — BACKGROUND & DECISIONS

---

## §1 How we got here — the decision journey

The mobile app was not the original plan. It evolved through ten stages.

### Stage 1 — Premium themes for web and mobile

Initial question: which premium themes to buy. Surveyed ThemeForest, Instamobile, CodeCanyon, UI8.  
Finding: theme bloat is risky for a solo team. No purchase made.

### Stage 2 — Web rewrite Astro → React

Tangent: rewriting the portal from Astro to React (Vite SPA). Explored the server-first → client-rendered mental shift and SEO implications. Not pursued — Astro portal was in good shape.

### Stage 3 — React styling recommendation

Reviewed React UI ecosystems. Top recommendation was Tailwind Plus + shadcn/ui. Irrelevant once we pivoted to mobile.

### Stage 4 — Pivot: we are actually building a mobile app

Sharp pivot. The real target was MyGate-style app UX: warm cream background, navy primary, rounded cards. Tailwind Plus dropped out entirely.

### Stage 5 — Native iOS + Android separately

Initial direction: SwiftUI on iOS, Jetpack Compose on Android — two separate codebases. Problems: no Tailwind-equivalent for native, fragmented ecosystem, 6+ months to ship v1, permanent 2× maintenance.

### Stage 6 — Figma deep-dive

Evaluated Figma (Untitled UI, \$129 lifetime). For a solo non-designer engineer, Figma overhead outweighs benefit unless someone on the team does UI design.

### Stage 7 — The 20-day timeline

Timeline stated as 20 days to v1. Native dual-codebase was immediately ruled out — even SwiftUI alone cannot produce a 28-module app in 20 days from a standing start.

### Stage 8 — Cross-platform comparison

Three contenders: .NET MAUI (small ecosystem), Flutter/Dart (large ecosystem, C#-friendly), React Native/TypeScript (largest community, JS-heavy).

### Stage 9 — Native vs Flutter: the 40-question grilling

40 questions across time, learning curve, tooling, code reuse, operational overhead. Every answer reinforced cross-platform. No app feature required native-only capability.

### Stage 10 — Flutter vs React Native: the final fork

Our developer background is .NET/C#. Dart maps cleanly to C# (classes, async/await, generics, null safety). Flutter won on the developer-profile axis. Full reasoning is in [mobile-stack-decision.md](#).

---

## §2 Why Flutter + Dart — the full reasoning

Flutter vs native

| Concern   | Native (iOS + Android) | Flutter |
|-----------|------------------------|---------|
| Codebases | 2                      | 1       |

| Concern                | Native (iOS + Android) | Flutter              |
|------------------------|------------------------|----------------------|
| New languages to learn | Swift + Kotlin         | Dart only            |
| Time to ship v1        | 6+ months              | 8–12 weeks           |
| Maintenance forever    | 2×                     | 1×                   |
| Hot reload             | Separate, flaky        | Sub-second, stateful |
| Design system          | Define twice           | Define once          |

## Flutter vs React Native (for a C# developer)

| Axis             | Flutter                   | React Native                           |
|------------------|---------------------------|----------------------------------------|
| Language from C# | Dart — very C#-like       | TypeScript + JSX — larger mental shift |
| Rendering        | Own engine, pixel-perfect | Bridges to native widgets              |
| Hot reload       | Sub-second, stateful      | Good but slightly less sharp           |
| Toolchain        | One unified framework     | Assembled from multiple pieces         |

## Tradeoffs accepted

- App binary ~5 MB larger (Flutter bundles its own runtime). Irrelevant for a resident portal.
- Cutting-edge iOS features (Live Activities, Dynamic Island) lag by weeks. Not in scope.
- iOS scroll physics very close but not 100% identical. Users don't notice.
- Smaller hiring pool than React Native. Accepted.

# PART II — FLUTTER & DART FOR THE NEWCOMER

## §3 Dart language primer — everything you need to know

If you know C#, Dart will feel immediately familiar. This section maps concepts you already know.

### Variables and types

```
// Dart is strongly typed (like C#) but has type inference
var name = 'Subbu';           // inferred as String
String city = 'Hyderabad';    // explicit type
int count = 42;
double price = 9.99;
bool isActive = true;

// final = readonly after assignment (like C# readonly)
final created = DateTime.now();

// const = compile-time constant (like C# const)
```

```
const maxItems = 100;

// Null safety – Dart requires you to be explicit about nulls
String? maybeNull = null;    // ? means nullable
String definitelyNotNull = 'hello'; // cannot be null
```

## Null safety operators

```
String? name = null;

// Safe access – returns null instead of crashing
int? length = name?.length;

// Null coalescing (like C# ??)
String display = name ?? 'Unknown';

// Force unwrap – crashes if null (use only when you are certain)
String forced = name!;

// Null-coalescing assignment (like C# ??=)
name ??= 'Default';
```

## Functions

```
// Regular function
int add(int a, int b) => a + b;    // arrow syntax for single expression

// Named parameters (common in Flutter widgets)
void greet({required String name, String greeting = 'Hello'}) {
  print('$greeting, $name');
}
greet(name: 'Subbu');
greet(name: 'Subbu', greeting: 'Namaste');

// Async/await (identical to C#)
Future<String> fetchData() async {
  final result = await someAsyncOperation();
  return result;
}

// Anonymous functions (lambdas)
final items = [1, 2, 3];
final doubled = items.map((x) => x * 2).toList();
```

## Classes

```

class Profile {
  final String id;           // readonly after construction (like C# get-only)
  final String? unitNumber; // nullable
  String portalRole;        // mutable

  // Constructor with named parameters
  Profile({
    required this.id,
    this.unitNumber,
    this.portalRole = 'member', // default value
  });

  // Named factory constructor (like a static factory method)
  factory Profile.fromJson(Map<String, dynamic> json) {
    return Profile(
      id: json['id'] as String,
      unitNumber: json['unit_number'] as String?,
      portalRole: json['portal_role'] as String? ?? 'member',
    );
  }

  // Computed property (like C# get-only property)
  bool get isExec => ['executive', 'secretary', 'president'].contains(portalRole);
}

```

## Collections

```

// List (like C# List<T>)
final notices = <String>['Notice 1', 'Notice 2'];
notices.add('Notice 3');
final first = notices.first;
final filtered = notices.where((n) => n.contains('1')).toList();

// Map (like C# Dictionary<K,V>)
final data = <String, dynamic>{'id': '123', 'name': 'Subbu'};
final name = data['name'] as String?;

// Spread operator (useful for building widget lists)
final extra = ['item3'];
final all = ['item1', 'item2', ...extra]; // ['item1', 'item2', 'item3']

// Collection if (conditional item in list)
final showAdmin = true;
final items = [
  'Home',
  'Profile',

```

```
    if (showAdmin) 'Admin', // only included when showAdmin is true
  ];
```

## Enums

```
enum AuthStatus { unknown, authenticated, unauthenticated }

// Usage
AuthStatus status = AuthStatus.authenticated;
if (status == AuthStatus.authenticated) { ... }
```

## Streams and async streams

```
// Stream is like IEnumerable in C# or RxJS Observable
// Supabase auth state is a Stream<AuthState>
Stream<AuthState> get authChanges => supabase.auth.onAuthStateChange;

// Listen to a stream
final subscription = authChanges.listen((event) {
  print('Auth changed: ${event.session}');
});
// Cancel when done
subscription.cancel();
```

## String interpolation

```
final name = 'Subbu';
final greeting = 'Hello, $name'; // simple variable
final upper = 'Hello, ${name.toUpperCase()}'; // expression in braces
```

## Cascade notation

```
// Instead of:
final controller = TextEditingController();
controller.text = 'hello';
controller.selection = TextSelection.collapsed(offset: 5);

// You can chain with ..
final controller = TextEditingController()
  ..text = 'hello'
  ..selection = TextSelection.collapsed(offset: 5);
```

## §4 Flutter fundamentals — how the framework works

### Everything is a Widget

In Flutter, **everything you see on screen is a Widget**. A widget is just a Dart class that describes a part of the UI. Widgets are immutable — they describe what to draw, they do not draw themselves. Flutter's rendering engine does the drawing.

There are two types of widgets:

**StatelessWidget** — UI that does not change after it is built. Receives data through the constructor.

```
class NoticeTile extends StatelessWidget {
  final String title;
  final DateTime publishedAt;

  const NoticeTile({super.key, required this.title, required
this.publishedAt});

  @override
  Widget build(BuildContext context) {
    return ListTile(
      title: Text(title),
      subtitle: Text(publishedAt.toString()),
    );
  }
}
```

**StatefulWidget** — UI that can change (e.g., a form, a toggle, a countdown). Has a **State** object that holds mutable data.

```
class CounterWidget extends StatefulWidget {
  const CounterWidget({super.key});

  @override
  State<CounterWidget> createState() => _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {
  int _count = 0;

  void _increment() {
    setState(() { // setState tells Flutter to re-build this widget
      _count++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(children: [
```

```

    Text('Count: $_count'),
    ElevatedButton(onPressed: _increment, child: const Text('+')),
  ]);
}
}

```

## The widget tree

A Flutter app is a tree of widgets. Every widget has a `build()` method that returns other widgets:

```

MaterialApp
├── Scaffold
│   ├── AppBar
│   │   └── Text('Notices')
│   └── ListView
│       ├── NoticeTile
│       ├── NoticeTile
│       └── NoticeTile

```

## Layout widgets

| Widget                             | What it does                         | HTML/CSS equivalent                                        |
|------------------------------------|--------------------------------------|------------------------------------------------------------|
| <code>Column</code>                | Vertical stack                       | <code>flex-direction: column</code>                        |
| <code>Row</code>                   | Horizontal stack                     | <code>flex-direction: row</code>                           |
| <code>Stack</code>                 | Absolute overlay                     | <code>position: relative + position: absolute</code>       |
| <code>Container</code>             | Box with padding, margin, decoration | <code>&lt;div&gt;</code> with styles                       |
| <code>SizedBox</code>              | Fixed-size space                     | <code>width/height</code> with nothing inside              |
| <code>Expanded</code>              | Fill remaining space                 | <code>flex: 1</code>                                       |
| <code>Padding</code>               | Add padding                          | <code>padding</code>                                       |
| <code>Center</code>                | Centre child                         | <code>margin: auto</code>                                  |
| <code>ListView</code>              | Scrollable list                      | <code>&lt;ul&gt;</code> or <code>overflow-y: scroll</code> |
| <code>GridView</code>              | Scrollable grid                      | CSS Grid                                                   |
| <code>SingleChildScrollView</code> | Scrollable container                 | <code>overflow-y: scroll</code>                            |

## Common display widgets

```

Text('Hello') // Display text
Text('Big', style: TextStyle(fontSize: 24)) // Styled text

```

```

Icon(Icons.home) // Material icon
Image.network('https://...') // Image from URL
CircularProgressIndicator() // Loading spinner
Divider() // Horizontal line

```

## Input widgets

```

TextFormField(
  controller: _myController,
  decoration: InputDecoration(labelText: 'Email', hintText:
'you@example.com'),
  keyboardType: TextInputType.emailAddress,
  onChanged: (value) { /* called on every keystroke */ },
)

ElevatedButton(
  onPressed: () { /* action */ },
  child: Text('Submit'),
)

Checkbox(value: _checked, onChanged: (v) => setState(() => _checked = v!))
Switch(value: _on, onChanged: (v) => setState(() => _on = v))

```

## BuildContext

**BuildContext** is a reference to the location of a widget in the tree. It is used to:

- Navigate (`context.go('/path')`)
- Show snackbars (`ScaffoldMessenger.of(context).showSnackBar(...)`)
- Access the theme (`Theme.of(context).textTheme`)
- Access media query info (`MediaQuery.of(context).size.width`)

You get it as the first parameter of `build()`.

## The Material 3 design system

This app uses Material 3 (`useMaterial3: true`). Material 3 provides:

- **Scaffold** — page structure (appBar, body, bottomNavigationBar, floatingActionButton)
- **AppBar** — top navigation bar
- **NavigationBar** — bottom tab bar (replaces **BottomNavigationBar**)
- **Card** — elevated content card
- **ElevatedButton**, **OutlinedButton**, **TextButton** — buttons
- **TextFormField** / **TextField** — text input
- **Dialog**, **AlertDialog** — modal dialogs
- **SnackBar** — toast-style notification
- **Drawer** — side panel
- **BottomSheet** — slide-up panel

## Understanding `async` in Flutter

Flutter's UI runs on the main thread. All async operations (network calls, file I/O) must be awaited or run in a `Future` — they do not block the UI.

```
// Pattern for a button that triggers an async operation
bool _loading = false;

Future<void> _submit() async {
  setState(() => _loading = true);
  try {
    await someAsyncOperation();
    // Success
  } catch (e) {
    // Show error
  } finally {
    setState(() => _loading = false);
  }
}

ElevatedButton(
  onPressed: _loading ? null : _submit, // null disables the button
  child: _loading
    ? const CircularProgressIndicator(color: Colors.white)
    : const Text('Submit'),
)
```

---

## §5 Key packages used in this project — why and how

### supabase\_flutter ^2.9.0

The official Supabase client for Flutter. Provides database queries, authentication, realtime subscriptions, and storage. Initialised once in `main.dart`.

```
// Access the client anywhere
final client = Supabase.instance.client;

// Query
final data = await client.from('notices').select().eq('society_id', id);

// Auth
await client.auth.signInWithOtp(email: 'user@example.com');
```

Official docs: <https://supabase.com/docs/reference/dart>

### flutter\_riverpod ^2.6.1 + riverpod\_annotation ^2.6.1

State management. Think of it as dependency injection + reactive state. A "provider" is a piece of shared state or a cached async result that any widget can subscribe to. When the data changes, all widgets that watch it rebuild automatically.

Why Riverpod over other options:

- **vs setState**: setState is local to one widget. Riverpod shares state across the whole app.
- **vs Provider** (predecessor): Riverpod is compile-time type safe. Provider can crash at runtime.
- **vs BLoC**: Riverpod has far less boilerplate.
- **vs GetX**: Riverpod is more testable and has no magic globals.

Riverpod docs: [https://riverpod.dev/docs/introduction/why\\_riverpod](https://riverpod.dev/docs/introduction/why_riverpod)

## go\_router ^15.1.2

Declarative URL-based navigation. Before go\_router, Flutter navigation used an imperative stack (`Navigator.push(context, route)`). go\_router introduces URL paths like a web app, which enables:

- Deep links (open a specific screen from a notification)
- Back-button behaviour matching user expectations
- Auth-aware redirects (send unauthenticated users to login)

go\_router docs: [https://pub.dev/packages/go\\_router](https://pub.dev/packages/go_router)

## google\_fonts ^6.2.1

Downloads Inter and Poppins font files at runtime from Google Fonts CDN on first launch, then caches them permanently. No TTF files need to be bundled in the app binary.

```
Text('Hello', style: GoogleFonts.poppins(fontSize: 18, fontWeight:
FontWeight.w700))
```

## flutter\_dotenv ^5.2.1

Loads a .env file that is embedded as an app asset. The .env file is read once at startup (`await dotenv.load()`) and its values are available via `dotenv.env['KEY']`.

Why not hardcoded? So the same app source code can connect to staging vs production Supabase by changing one file, without code changes.

## qr\_flutter ^4.1.0

Generates QR code images. Used for visitor passes — the QR contains a JSON payload with the pass ID and token that a guard's scanner reads.

```
QrImageView(
  data: '{"pass_id":"uuid","token":"secret"}',
```

```
size: 200.0,
)
```

### mobile\_scanner ^7.0.1

Scans QR codes and barcodes using the device camera. Used for guards scanning visitor passes at the gate. Not yet wired up in the current build — see §30.

### flutter\_secure\_storage ^9.2.4

Stores sensitive data securely: Keychain on iOS, EncryptedSharedPreferences on Android. Used internally by `supabase_flutter` to persist the auth session token. You do not call it directly in most cases.

### shimmer ^3.0.0

Creates skeleton/placeholder loading effects (grey animated shimmer) for list screens while data is loading. Not yet applied to screens — see §40.

### intl ^0.20.2

Internationalisation and date/number formatting.

```
import 'package:intl/intl.dart';

final formatted = DateFormat('dd MMM yyyy').format(DateTime.now()); // "15
May 2026"
final currency = NumberFormat.currency(locale: 'en_IN', symbol:
'₹').format(1500); // "₹1,500.00"
```

### timeago ^3.7.0

Human-readable relative times.

```
import 'package:timeago/timeago.dart' as timeago;

timeago.format(DateTime.now().subtract(Duration(minutes: 5))); // "5
minutes ago"
```

### build\_runner + riverpod\_generator

Development-only tools that generate boilerplate Dart code. Run `dart run build_runner build` after changing any file with `@riverpod` annotations. The output `.g.dart` files are committed to git so other developers do not need to run `build_runner` just to compile the project.

---

## PART III — SETTING UP FROM ZERO

---

---

## §6 Machine setup — install everything

**Required OS:** macOS (required for iOS builds and Xcode). Windows/Linux can build Android only.

Work through each step in order. Do not skip **flutter doctor** — it tells you exactly what is missing.

### Step 1 — Install Homebrew (macOS package manager)

Homebrew is required for several steps. If you already have it, skip.

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Verify:

```
brew --version
```

### Step 2 — Install Flutter SDK

```
# Option A — via Homebrew (easiest, manages updates)
brew install --cask flutter

# Option B — manual install
# 1. Download from https://docs.flutter.dev/get-started/install/macos/mobile-ios
# 2. Extract to ~/development/flutter
# 3. Add to PATH in ~/.zshrc:
echo 'export PATH="$HOME/development/flutter/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Verify:

```
flutter --version
# Expected output: Flutter 3.x.x • channel stable • Dart 3.x.x
```

### Step 3 — Install Xcode (iOS + macOS builds)

1. Open the Mac App Store
2. Search "Xcode" and install it (it is ~14 GB — this takes a while)
3. Once installed, run:

```
sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer
sudo xcodebuild -runFirstLaunch
# Accept the license agreement when prompted
```

Verify:

```
xcode-select -p
# Expected: /Applications/Xcode.app/Contents/Developer
xcodebuild -version
# Expected: Xcode 15.x or 16.x
```

## Step 4 — Install CocoaPods (iOS dependency manager)

```
# Via Homebrew (preferred – avoids system Ruby conflicts)
brew install cocoapods

# Verify
pod --version
# Expected: 1.x.x
```

## Step 5 — Install Android Studio (Android builds)

1. Download from <https://developer.android.com/studio>
2. Run the installer. During setup, tick: Android SDK, Android SDK Platform-Tools, Android Emulator
3. After installation, open Android Studio → More Actions → SDK Manager
4. Under SDK Platforms: install Android 14 (API 34) or the latest
5. Under SDK Tools: ensure these are checked:
  - Android SDK Build-Tools
  - Android Emulator
  - Android SDK Platform-Tools
  - Intel x86 Emulator Accelerator (HAXM) — if on Intel Mac
  - Android SDK Command-line Tools

Set environment variables in `~/.zshrc`:

```
export ANDROID_HOME=$HOME/Library/Android/sdk
export PATH=$ANDROID_HOME/tools:$ANDROID_HOME/platform-
tools:$ANDROID_HOME/cmdline-tools/latest/bin:$PATH
```

Reload:

```
source ~/.zshrc
```

Accept Android licences:

```
flutter doctor --android-licenses  
# Type 'y' and press Enter for each prompt
```

## Step 6 — Install VS Code (recommended IDE)

1. Download from <https://code.visualstudio.com>
2. Install these extensions (press Cmd+Shift+X, search each):
  - **Flutter** (by Dart Code) — required
  - **Dart** (by Dart Code) — required
  - **Error Lens** (by Alexander) — shows errors inline
  - **Pubspec Assist** (by Jeroen Meijer) — adds packages inline

## Step 7 — Run flutter doctor

```
flutter doctor -v
```

You should see all green checkmarks. Common issues and fixes:

| Issue                                               | Fix                                                                                    |
|-----------------------------------------------------|----------------------------------------------------------------------------------------|
| Xcode – develop for iOS and macOS (Xcode x.x):<br>x | Run <code>sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer</code> |
| CocoaPods not installed                             | Run <code>brew install cocoapods</code>                                                |
| Android toolchain – missing SDK                     | Open Android Studio → SDK Manager → install                                            |
| Android licenses not accepted                       | Run <code>flutter doctor --android-licenses</code>                                     |
| VS Code is not installed or missing                 | Install VS Code (warning only, not an error)                                           |
| Unable to get list of devices                       | Connect a device or start a simulator first                                            |

## Step 8 — Clone the repository

```
git clone https://github.com/utamacs/utamacs-website.git
cd utamacs-website/mobile
```

## Step 9 — Get Flutter packages

```
flutter pub get
```

This downloads all packages listed in `pubspec.yaml` into the `.dart_tool/` cache.

## Step 10 — Create the .env file

```
cp .env.example .env
# Edit .env with a text editor and fill in real values:
# SUPABASE_URL – from Supabase dashboard → Project Settings → API
# SUPABASE_ANON_KEY – from Supabase dashboard → Project Settings → API
# (anon/public)
# SOCIETY_ID – the UUID of the UTAMACS society row in the societies table
```

## Step 11 — Run code generation

```
dart run build_runner build --delete-conflicting-outputs
```

This generates the `.g.dart` files needed by Riverpod. If they are already present and up to date (they are committed to git), this step can be skipped.

## Step 12 — Verify everything works

```
flutter run -d macos
```

The app should launch on your Mac. If you see the UTAMACS login screen, everything is working.

---

## §7 Project creation and initial scaffold

This section documents how the project was originally created. You only need this if you are starting a brand-new Flutter project. If you are continuing work on the existing project, skip to §9.

```
# From the monorepo root
flutter create \
  --org org.utamacs \
```

```
--project-name utamacs_portal \  
--platforms android,ios \  
mobile
```

Flag meanings:

- `--org org.utamacs` — reverse-domain package prefix (like Java package names)
- `--project-name utamacs_portal` — the Dart package name (snake\_case, no hyphens)
- `--platforms android,ios` — only generate Android and iOS platform code (omits web, desktop)
- `mobile` — the directory name

Adding macOS for development testing (done later):

```
cd mobile  
flutter create --platforms macos .
```

---

## §8 Environment configuration (.env)

Why .env instead of hardcoding

The `.env` file separates configuration from code. The same Dart source compiles to staging or production just by changing the `.env` values. It also prevents secrets from being committed to git.

The `.env.example` (committed — safe, no real secrets)

```
SUPABASE_URL=https://your-project-ref.supabase.co  
SUPABASE_ANON_KEY=your-anon-key-here  
SOCIETY_ID=00000000-0000-0000-0000-000000000001
```

How .env is loaded

The `.env` file is declared as an asset in `pubspec.yaml`:

```
flutter:  
  assets:  
    - assets/images/  
    - .env          # embedded in app binary as an asset
```

At startup in `main.dart`:

```
await dotenv.load(); // reads assets/.env (bundled inside the binary)
```

After loading, values are available via:

```
import 'package:flutter_dotenv/flutter_dotenv.dart';

dotenv.env['SUPABASE_URL'] // returns the value or null
dotenv.env['SUPABASE_URL']! // throws if null (use when you know it's
set)
dotenv.env['SOCIETY_ID'] ?? 'default-uuid' // fallback value
```

Security note — what is safe to put in .env

#### Safe to include:

- **SUPABASE\_URL** — the project URL (not secret)
- **SUPABASE\_ANON\_KEY** — the anonymous/public key. This is designed to be in client apps. Supabase Row Level Security (RLS) enforces access control, so even if someone decompiles the app and extracts this key, they can only access data that RLS permits.
- **SOCIETY\_ID** — a UUID, not a secret

#### Never put in .env for a mobile app:

- **SUPABASE\_SERVICE\_ROLE\_KEY** — this bypasses RLS entirely and gives full database access
- Any private key, certificate, or signing credential

How .env differs between environments

For production, keep the **.env** you use for release builds in a secure location (1Password, GitHub Actions secrets, etc.). Never commit the real **.env** to git.

## PART IV — CODEBASE DEEP-DIVE

### §9 Repository structure

```
utamacs-website/
├── mobile/                                ← Flutter app root
│   ├── lib/
│   │   ├── main.dart                    ← Entry point (4 lines)
│   │   ├── app.dart                    ← Router + shell + tab bar
│   │   └── core/
│   │       ├── constants/
│   │       │   └── supabase.dart        ← Reads SUPABASE_URL, ANON_KEY,
SOCIETY_ID from .env
│   │       ├── theme/
│   │       │   ├── app_theme.dart      ← All design tokens + ThemeData
│   │       │   └── utils/
│   │       │       └── formatters.dart  ← Date, currency, phone formatters
```

|          |                     |                                       |
|----------|---------------------|---------------------------------------|
|          | features/           |                                       |
|          | auth/               | ← Login, OTP, session, profile load   |
|          | dashboard/          | ← Home screen                         |
|          | notices/            |                                       |
|          | visitors/           |                                       |
|          | complaints/         |                                       |
|          | finance/            |                                       |
|          | events/             |                                       |
|          | polls/              |                                       |
|          | community/          |                                       |
|          | documents/          |                                       |
|          | facilities/         |                                       |
|          | parking/            |                                       |
|          | maids/              |                                       |
|          | members/            |                                       |
|          | gallery/            |                                       |
|          | water_tankers/      |                                       |
|          | vendors/            |                                       |
|          | feedback/           |                                       |
|          | snags/              |                                       |
|          | security_patrol/    |                                       |
|          | policies/           |                                       |
|          | register/           |                                       |
|          | agm/                |                                       |
|          | tenant_kyc/         |                                       |
|          | hoto/               |                                       |
|          | letters/            |                                       |
|          | analytics/          |                                       |
|          | staff_management/   |                                       |
|          | notifications_list/ |                                       |
|          | profile/            |                                       |
|          | services/           | ← "All Services" grid                 |
|          | shared/             |                                       |
|          | models/             |                                       |
|          | profile.dart        | ← Profile model used everywhere       |
|          | widgets/            |                                       |
|          | app_card.dart       |                                       |
|          | empty_state.dart    |                                       |
|          | status_badge.dart   |                                       |
| usually) | android/            | ← Android platform files (do not edit |
|          | app/                |                                       |
|          | build.gradle.kts    | ← App ID, min SDK, target SDK         |
| usually) | ios/                | ← iOS platform files (do not edit     |
|          | Podfile             | ← CocoaPods config                    |
|          | Runner/             |                                       |
|          | Info.plist          | ← App permissions, bundle ID          |
|          | macos/              | ← macOS (dev testing only)            |
|          | assets/             |                                       |
|          | images/             | ← App images (logo, etc.)             |
|          | .env                | ← GITIGNORED – real secrets           |
|          | .env.example        | ← Committed – placeholder values only |
|          | pubspec.yaml        | ← Package dependencies + asset        |

```

declarations
├── analysis_options.yaml      ← Dart linting rules
├── design/
│   ├── mobile-stack-decision.md
│   └── MOBILE-RUNBOOK.md      ← this file

```

## The feature folder pattern

Every feature follows the same three-layer structure:

```

features/{feature_name}/
├── data/
│   ├── {feature}_repository.dart      ← Model classes + Supabase queries
│   └── {feature}_repository.g.dart    ← Auto-generated (do not edit)
├── domain/                            ← Only for complex state (auth uses
this)
│   ├── {feature}_notifier.dart
│   └── {feature}_notifier.g.dart      ← Auto-generated
├── presentation/
│   └── screens/
│       ├── {feature}_screen.dart      ← Main list/overview screen
│       └── {feature}_detail_screen.dart ← Optional detail screen

```

Simple features (notices, finance, gallery) skip the domain layer. Only **auth** has a domain layer because its state (unauthenticated / loading / authenticated + profile) needs to be observable app-wide.

## §10 Architecture — how the app is built

Entry point (**main.dart**)

```

void main() async {
  WidgetsFlutterBinding.ensureInitialized(); // (1) init platform channels
  await dotenv.load();                       // (2) load .env file
  await Supabase.initialize(                 // (3) init Supabase SDK
    url: env.supabaseUrl,
    anonKey: env.supabaseAnonKey,
  );
  runApp(const ProviderScope(child: UtamacsApp())); // (4) start app
}

```

**ProviderScope** is the Riverpod container. All providers live inside it. It wraps the entire app so any widget can access any provider.

The router (**app.dart**)

The **GoRouter** instance is built once (**final \_router = \_buildRouter()**). It:

1. Listens to session changes via `_AuthNotifier` (a `ChangeNotifier` that wraps `supabase.auth.onAuthStateChange`). Every session change triggers a router re-evaluation.
2. Runs a `redirect` function on every navigation:

```
redirect: (context, state) {  
  final isLoggedIn = Supabase.instance.client.auth.currentSession !=  
  null;  
  if (!isLoggedIn && state.matchedLocation != '/login') return  
  '/login';  
  if (isLoggedIn && state.matchedLocation == '/login') return '/';  
  return null; // no redirect  
}
```

3. Wraps all authenticated screens in a `ShellRoute` which injects `_AppShell` (the bottom nav bar).

### Bottom navigation shell (`_AppShell`)

`_AppShell` is a `StatelessWidget` that renders the `NavigationBar` at the bottom and the current page widget above it. The current tab is determined by checking which path the current `location` starts with:

```
int get _currentIndex {  
  if (location.startsWith('/notices')) return 1;  
  if (location.startsWith('/visitors')) return 2;  
  if (location.startsWith('/services')) return 3;  
  if (location.startsWith('/profile')) return 4;  
  return 0; // home  
}
```

This means `/visitors`, `/visitors/new`, `/visitors/pass/uuid` all keep the Visitors tab highlighted.

### Data flow

```

User action (tap button)
  ↓
Widget calls: ref.watch(provider) or ref.read(provider.notifier).method()
  ↓
Riverpod Provider / Notifier
  ↓
Repository method (Dart class)
  ↓
Supabase client (HTTP to Supabase PostgREST API)
  ↓
Response parsed into model class
  ↓
Riverpod state updated → all watching widgets rebuild

```

## §11 Design system — tokens mapped from the web portal

The web portal uses Tailwind CSS tokens defined in `tailwind.config.cjs`. Every token has been exactly mapped to a Dart `const Color` in `lib/core/theme/app_theme.dart`.

### Colour tokens

```

// In app_theme.dart – match these to tailwind.config.cjs exactly
const kPrimary600    = Color(0xFF1E3A8A); // primary-600
const kPrimary100    = Color(0xFFDBEAFE); // primary-100
const kPrimary50     = Color(0xFFEFF6FF); // primary-50
const kSecondary500  = Color(0xFF10B981); // secondary-500
const kAccent500     = Color(0xFFF59E0B); // accent-500
const kTextPrimary   = Color(0xFF111827); // text-primary
const kTextSecondary = Color(0xFF4B5563); // text-secondary
const kBorderLight   = Color(0xFFE5E7EB); // border-light
const kSectionAlt    = Color(0xFFF8FAFC); // section-alt
const kRed600        = Color(0xFFDC2626); // red-600
const kBgWarm        = Color(0xFFF5F0EB); // warm cream (MyGate-inspired)

```

### Typography mapping

| Portal Tailwind class                               | Flutter equivalent                                                                    |
|-----------------------------------------------------|---------------------------------------------------------------------------------------|
| font-poppins font-bold<br>text-2xl text-primary-600 | GoogleFonts.poppins(fontWeight: FontWeight.w700,<br>fontSize: 22, color: kPrimary600) |
| font-inter text-sm text-text-secondary              | GoogleFonts.inter(fontSize: 12, color:<br>kTextSecondary)                             |
| font-inter font-medium<br>text-text-primary         | GoogleFonts.inter(fontWeight: FontWeight.w500,<br>color: kTextPrimary)                |

## Layout mapping

| Portal Tailwind                      | Flutter                                                                           |
|--------------------------------------|-----------------------------------------------------------------------------------|
| <code>rounded-xl</code> (12px)       | <code>BorderRadius.circular(12)</code>                                            |
| <code>rounded-2xl</code> (16px)      | <code>BorderRadius.circular(16)</code>                                            |
| <code>rounded-full</code>            | <code>BorderRadius.circular(999)</code> or <code>BoxShape.circle</code>           |
| <code>p-4</code> (16px)              | <code>padding: const EdgeInsets.all(16)</code>                                    |
| <code>px-4 py-3</code>               | <code>EdgeInsets.symmetric(horizontal: 16, vertical: 12)</code>                   |
| <code>shadow-soft</code>             | <code>BoxShadow(blurRadius: 8, offset: Offset(0,2), color: Colors.black12)</code> |
| <code>flex items-center gap-3</code> | <code>Row(children: [..., SizedBox(width: 12), ...])</code>                       |

## §12 Authentication flow — step by step

### The login screen

1. User enters email address
2. `_sendCode()` called → `supabase.auth.signInWithOtp(email: email, shouldCreateUser: false)`
  - `shouldCreateUser: false` means only pre-existing accounts can sign in. A new email address gets an error, not a new account.
3. Supabase sends a 6-digit code to the email
4. UI switches to OTP entry field
5. User enters the 6-digit code
6. `_verifyCode()` called → `supabase.auth.verifyOTP(email, token, type: OtpType.email)`
7. Supabase validates and returns a `Session` (access token + refresh token)
8. `_AuthNotifier` receives the `authStateChange` event → calls `fetchProfile()` → loads user's `Profile`
9. `GoRouter.refreshListenable` fires → redirect re-evaluates → has session → goes to /

### Token persistence

Supabase Flutter stores the session token in:

- iOS: Keychain (encrypted, survives app deletion only if iCloud Keychain is on)
- Android: EncryptedSharedPreferences

When the app starts next time, Supabase automatically restores the session from storage. The user stays logged in indefinitely until they sign out or the refresh token expires.

### Profile loading

```
Future<Profile?> fetchProfile() async {
  final uid = currentUser?.id;
  if (uid == null) return null;
  final data = await _client
    .from('profiles')
    .select('*', units(unit_number, block)) // PostgREST FK join
    .eq('id', uid)
    .eq('society_id', env.societyId)
    .maybeSingle(); // returns null if not found, instead of throwing
  if (data == null) return null;
  return Profile.fromJson(data);
}
```

## Sign out

```
await supabase.auth.signOut();
// This clears the stored session. GoRouter's redirect fires and sends user
to /login.
```

## §13 State management with Riverpod

### The mental model

A **provider** is like a global variable with superpowers: it is lazy (created on first use), cached (same value returned to all users), automatically disposed when no longer needed, and observable (widgets rebuild when the value changes).

Think of it as a reactive cache for your data and state.

### Defining a simple data provider

```
// In notice_repository.dart
@riverpod // ← annotation triggers code generation
Future<List<Notice>> notices(NoticesRef ref) =>
  ref.watch(noticeRepositoryProvider).fetchNotices();
// This generates: noticesProvider
```

After running `build_runner`, you can use `noticesProvider` in any widget.

### Using a provider in a widget

```
class NoticesScreen extends ConsumerWidget {
  // ConsumerWidget instead of StatelessWidget – gives access to ref
  @override
```

```
Widget build(BuildContext context, WidgetRef ref) {
  final noticesAsync = ref.watch(noticesProvider);
  // ~~~~~
  // ref.watch subscribes this widget to the provider.
  // When noticesProvider changes, this widget rebuilds automatically.

  return noticesAsync.when(
    loading: () => const Center(child: CircularProgressIndicator()),
    error: (err, stack) => Center(child: Text('Failed to load: $err')),
    data: (notices) => ListView.builder(
      itemCount: notices.length,
      itemBuilder: (_, i) => Text(notices[i].title),
    ),
  );
}
```

## Reading a provider without subscribing

```
// ref.read – reads once, does not subscribe (widget won't rebuild on
// change)
// Use in event handlers (button press, etc.)
onPressed: () {
  ref.read(authNotifierProvider.notifier).signOut();
}
```

## Invalidating (refreshing) a provider

```
// Force a re-fetch (use in pull-to-refresh, after mutations)
ref.invalidate(noticesProvider);
```

## The AuthNotifier (complex state example)

**AuthNotifier** is a **@riverpod** class (generates an **AsyncNotifier**-style state). It:

- Holds **AuthState** (status + loaded profile)
- Listens to **supabase.auth.onAuthStateChange** stream
- Exposes **sendEmail0tp**, **verifyEmail0tp**, **signOut** methods

Any widget can read the current user's profile:

```
final authState = ref.watch(authNotifierProvider);
final profile = authState.profile;
final isExec = profile?.isExec ?? false;
```

## Running code generation

After every change to a file with `@riverpod`:

```
cd mobile
dart run build_runner build --delete-conflicting-outputs
```

Watch mode (rebuilds on every file save during development):

```
dart run build_runner watch --delete-conflicting-outputs
```

The generated `.g.dart` files are committed to git. Other developers cloning the repo can compile without running `build_runner`.

---

## §14 Supabase integration

### Initialisation

Done once in `main.dart`. After this, `Supabase.instance.client` is a global singleton available anywhere.

### Query patterns

```
final client = Supabase.instance.client;

// SELECT with filter and order
final data = await client
  .from('notices')
  .select('id, title, published_at, is_pinned')
  .eq('society_id', env.societyId)
  .eq('status', 'published')
  .order('is_pinned', ascending: false)
  .order('published_at', ascending: false)
  .limit(30);
// data is List<Map<String, dynamic>>

// SELECT with FK join (PostgreSQL syntax)
final profile = await client
  .from('profiles')
  .select('*', units(unit_number, block)) // joins units table via FK
  .eq('id', userId)
  .maybeSingle(); // returns null if no row found

// INSERT
final newRow = await client
  .from('visitor_pre_approvals')
  .insert({'visitor_name': name, 'society_id': societyId, ...})
```

```

        .select()    // returns the inserted row
        .single();   // returns one row (throws if zero or multiple)

// UPDATE
await client
    .from('complaints')
    .update({'status': 'resolved'})
    .eq('id', complaintId)
    .eq('society_id', societyId); // always scope by society_id for safety

// DELETE
await client
    .from('visitor_pre_approvals')
    .delete()
    .eq('id', passId);

```

## RLS (Row Level Security)

All tables have RLS enabled on the Supabase side. The mobile app uses the `anon_key`, which means all queries automatically run as the authenticated user. RLS policies ensure:

- A member can only read their society's data
- A member can only insert/update their own records
- Admin operations require the exec role (enforced server-side)

You do not need to add extra WHERE clauses for ownership — RLS handles it. You DO need to always add `.eq('society_id', env.societyId)` for performance (uses the index) even though RLS would filter it anyway.

## Error handling

```

try {
    final data = await client.from('notices').select();
} on PostgreSQLException catch (e) {
    // Supabase/PostgREST error (e.g., RLS violation, constraint violation)
    print('DB error: ${e.message}, code: ${e.code}');
} on AuthException catch (e) {
    // Auth error
    print('Auth error: ${e.message}');
} catch (e) {
    // Network error, timeout, etc.
    print('Unknown error: $e');
}

```

## §15 Navigation model (go\_router)

Route table — all paths in the app

|                     |                                              |
|---------------------|----------------------------------------------|
| /login              | ← LoginScreen (outside shell, no bottom nav) |
| /                   | ← DashboardScreen                            |
| /notices            | ← NoticesScreen                              |
| /visitors           | ← VisitorsScreen                             |
| /services           | ← ServicesScreen (all modules grid)          |
| /profile            | ← ProfileScreen                              |
| /complaints         | ← ComplaintsScreen                           |
| /finance            | ← FinanceScreen                              |
| /events             | ← EventsScreen                               |
| /polls              | ← PollsScreen                                |
| /community          | ← CommunityScreen                            |
| /documents          | ← DocumentsScreen                            |
| /facilities         | ← FacilitiesScreen                           |
| /parking            | ← ParkingScreen                              |
| /maids              | ← MaidsScreen                                |
| /members            | ← MembersScreen                              |
| /notifications-list | ← NotificationsListScreen                    |
| /gallery            | ← GalleryScreen                              |
| /water-tankers      | ← WaterTankersScreen                         |
| /vendors            | ← VendorsScreen                              |
| /feedback           | ← FeedbackScreen                             |
| /snags              | ← SnagsScreen                                |
| /security-patrol    | ← SecurityPatrolScreen                       |
| /policies           | ← PoliciesScreen                             |
| /register           | ← RegisterScreen                             |
| /agm                | ← AgmScreen                                  |
| /tenant-kyc         | ← TenantKycScreen                            |
| /hoto               | ← HotoScreen                                 |
| /letters            | ← LettersScreen                              |
| /analytics          | ← AnalyticsScreen                            |
| /staff              | ← StaffScreen                                |

## Navigating in code

```
// Replace current screen (no back button – use for tab switches)
context.go('/notices');

// Push on top (back button returns to previous – use for detail screens)
context.push('/notices');

// Push with extra data (for passing objects without URL encoding)
context.push('/visitors/pass', extra: approvalObject);

// Go back
context.pop();
```

## Adding a new route

1. Create the screen widget in

`features/{name}/presentation/screens/{name}_screen.dart`

2. Add the route in `app.dart` inside the `ShellRoute.routes` list:

```
GoRoute(
  path: '/your-new-path',
  builder: (ctx, st) => const YourNewScreen(),
),
```

3. Add the navigation link wherever users should access it (Services grid, dashboard, etc.)

## §16 How Astro portal pages were ported to Flutter

The web portal has 28 modules as Astro SSR pages. Every module was rebuilt as a native Flutter screen — not WebViews. This is a true native rewrite.

The porting pattern

| Astro part                 | Flutter equivalent                                                        |
|----------------------------|---------------------------------------------------------------------------|
| Frontmatter auth check     | <code>GoRouter</code> redirect + <code>AuthNotifier</code>                |
| Frontmatter Supabase query | Repository class method + <code>@riverpod</code> provider                 |
| HTML template              | Flutter <code>Widget</code> tree                                          |
| Tailwind card classes      | <code>Container</code> with <code>BoxDecoration</code>                    |
| Vanilla JS interactivity   | <code>StatefulWidget.setState</code> or Riverpod notifier                 |
| Right-side detail drawer   | <code>Navigator.push(MaterialPageRoute(...))</code> — slides up on mobile |
| Toast notification         | <code>ScaffoldMessenger.of(context).showSnackBar(...)</code>              |
| Status badge span          | <code>StatusBadge</code> shared widget                                    |
| Empty state div            | <code>EmptyState</code> shared widget                                     |

What was ported as read-only (for now)

The 24 modules added in the big feature commit have list views. Most do not yet have write forms. Admin operations (create notice, approve complaint, etc.) remain web-only for now. The mobile app is primarily a resident-facing consumption tool in the current phase.

## §17 All 28 modules — what exists today

Tab bar (5 primary tabs)

| Route     | Screen                                                | What it shows                                                    |
|-----------|-------------------------------------------------------|------------------------------------------------------------------|
| /         | DashboardScreen                                       | Pinned notice banner, quick services grid, active visitor passes |
| /notices  | NoticesScreen + NoticeDetailScreen                    | Society circulars, pinned notices                                |
| /visitors | VisitorsScreen + PreApproveScreen + VisitorPassScreen | Pre-approvals, QR pass display                                   |
| /services | ServicesScreen                                        | 4-column grid of all 28 modules in 4 sections                    |
| /profile  | ProfileScreen                                         | Name, unit, role, sign out                                       |

## All modules via Services

**Resident Services:** Complaints · Finance · Events · Polls · Community · Documents · Facilities · Parking · Maids · Members · Notifications

**Society & Amenities:** Gallery · Water Tankers · Vendors · Feedback · Snags · Security Patrol

**Governance & Compliance:** Policies · Membership Registration · AGM · Tenant KYC

**Management:** HOTO · Letters · Analytics · Staff

## §18 Shared components

### AppCard

White card with 16px radius, 1px `kBorderLight` border, optional tap handler.

```
AppCard(
  onTap: () => context.go('/notices'),
  padding: const EdgeInsets.all(16),
  child: Text('Card content'),
)
```

### EmptyState

Centred placeholder for empty lists.

```
EmptyState(
  icon: Icons.notifications_none,
  title: 'No notices yet',
  description: 'New circulars will appear here.',
)
```

## StatusBadge

Colour-coded pill for status values.

```
StatusBadge(status: 'pending')    // amber pill
StatusBadge(status: 'approved')  // green pill
StatusBadge(status: 'rejected')  // red pill
StatusBadge(status: 'resolved')  // grey pill
```

## Profile model

Key properties and helpers:

```
profile.id           // Supabase user UUID
profile.fullName     // e.g., "Subramanyam"
profile.unitDisplay  // e.g., "B-101"
profile.portalRole   // 'member' | 'executive' | 'secretary' | 'president'
profile.isAdmin      // bool – orthogonal to portal_role
profile.isExec       // bool – true for
executive/secretary/president/admin
profile.isGuard      // bool – true for security_guard role
profile.displayName  // fullName ?? 'Resident'
```

---

# PART V — RUNNING & TESTING LOCALLY

---

## §19 Running the app — every platform

Check available devices first

```
flutter devices
# Shows all connected devices and running simulators/emulators
```

Example output:

```
Found 4 connected devices:
  macOS (desktop)           • macOS           • darwin-x64       • macOS 15.0
  iPhone 16 (mobile)       • <uuid>          • ios              • iOS 18.0
(simulator)
  sdk gphone64 x86 64 (mobile) • emulator       • android-x64     • API 34
  iPhone 14 Pro (mobile)   • <udid>          • ios              • iOS 17.5
(physical)
```

## Run on macOS (fastest for development)

```
flutter run -d macos
```

The app runs as a native macOS app on your laptop. Fastest feedback loop. Same code as iOS/Android.

## Run on iOS Simulator

```
flutter run -d iphone # matches any device with "iphone" in its name  
# or specify exactly:  
flutter run -d "iPhone 16 Pro"
```

## Run on Android Emulator

```
flutter run -d emulator-5554 # use the ID from flutter devices
```

## Run on a physical device

Connect via USB, then:

```
flutter run -d <device-id> # from flutter devices output
```

## Run with performance overlay (useful for spotting jank)

```
flutter run --profile -d macos  
# Then press P in the terminal to toggle the performance overlay
```

---

## §20 Using the iOS Simulator

### Starting the Simulator

#### From terminal:

```
open -a Simulator  
# Or open a specific device:  
xcrun simctl boot "iPhone 16 Pro"  
open -a Simulator
```

**From VS Code:** Bottom status bar → click the device name → select a simulator from the dropdown.

**From Xcode:** Xcode menu → Open Developer Tool → Simulator.

## Managing simulators

```
# List all available simulators
xcrun simctl list devices

# Create a new simulator (e.g., iPhone 15 with iOS 17)
xcrun simctl create "iPhone 15" "iPhone 15" "iOS-17-0"

# Delete a simulator
xcrun simctl delete <udid>

# Reset a simulator (clear all data – useful for testing first-run
experience)
xcrun simctl erase <udid>
```

## Useful simulator shortcuts

| Action               | Shortcut                                                     |
|----------------------|--------------------------------------------------------------|
| Home button          | Cmd+Shift+H                                                  |
| Lock screen          | Cmd+L                                                        |
| Rotate               | Cmd+Left / Cmd+Right                                         |
| Toggle dark mode     | Features → Toggle Appearance                                 |
| Screenshot           | Cmd+S                                                        |
| Open URL / deep link | <code>xcrun simctl openurl booted "utamacs://notices"</code> |

## Testing different screen sizes

Run the same code on multiple simulators to check layout on small phones (SE), standard (iPhone 15), and Pro Max (6.7"):

```
# Open multiple simulators and run on each
flutter run -d "iPhone SE (3rd generation)" # 4.7"
flutter run -d "iPhone 15" # 6.1"
flutter run -d "iPhone 15 Plus" # 6.7"
```

---

## §21 Using the Android Emulator

### Creating an Android Virtual Device (AVD)

1. Open Android Studio
2. Click "More Actions" → "Virtual Device Manager" (or Tools → Device Manager)
3. Click "+" → Create Virtual Device
4. Choose a hardware profile: "Pixel 7" is a good default
5. Select a system image: Android 14 (API 34) with Google APIs
6. Click Finish

## Starting the emulator

**From Android Studio:** Device Manager → click the ► play button next to your AVD.

**From terminal:**

```
# List all AVDs
emulator -list-avds

# Start a specific AVD
emulator -avd Pixel_7_API_34
```

## Useful emulator commands

```
# Cold boot (simulates first power-on, clears all state)
emulator -avd Pixel_7_API_34 -no-snapshot-load

# Install an APK directly
adb install build/app/outputs/flutter-apk/app-release.apk

# View device logs in real-time
adb logcat | grep flutter

# Clear app data (simulate fresh install)
adb shell pm clear org.utamacs.utamacs_portal

# Open a deep link
adb shell am start -a android.intent.action.VIEW -d "utamacs://notices"
```

## Testing different Android screen densities

Android devices vary in screen density (DPI). Test at:

- **mdpi** (160 DPI) — old/budget phones
- **xhdpi** (320 DPI) — common mid-range
- **xxxhdpi** (640 DPI) — flagship phones

---

## §22 Testing on a physical device

### iOS physical device

1. Connect iPhone via USB cable
2. On the phone: Settings → Privacy & Security → Developer Mode → turn on (reboot required)
3. When the "Trust This Computer?" prompt appears on the phone, tap "Trust"
4. In Xcode (one time): open `mobile/ios/Runner.xcworkspace`, go to Signing & Capabilities, select your Apple ID under Team
5. Run:

```
flutter run -d <iphone-udid>
```

**Note:** Running on a physical iPhone requires a free Apple Developer account for local development, or a paid account (\$99/year) for distribution.

### Android physical device

1. On the phone: Settings → About Phone → tap "Build number" 7 times to enable Developer Options
2. Settings → Developer Options → USB Debugging → enable
3. Connect via USB
4. Accept the "Allow USB Debugging?" prompt on the phone
5. Run:

```
flutter run -d <android-serial>
```

---

## §23 Hot reload, hot restart, and debugging

### Hot reload (r)

Press `r` in the terminal while the app is running. Flutter injects the changed code into the running app without losing state. The widget tree rebuilds with the new code. State (opened drawers, filled form fields, scroll position) is preserved.

#### When hot reload works:

- Changed widget `build()` methods
- Changed styles, colours, layouts
- Changed string literals

#### When hot reload does NOT work (use hot restart instead):

- Added new class (type system change)
- Changed `main()` or `initState()` logic
- Changed Riverpod provider definitions
- Changed routes

### Hot restart (R)

Press **R** (capital R) to fully restart the app, clearing all state but without rebuilding the binary. Faster than stopping and re-running.

## Flutter DevTools

The most powerful Flutter debugging tool. Start it while the app is running:

```
flutter pub global activate devtools
flutter pub global run devtools
```

Or in VS Code: Run → Start Debugging, then look for the DevTools button in the toolbar.

DevTools provides:

- **Widget Inspector** — click any widget on screen to see its properties in the tree
- **Performance** — flame graphs for CPU and rendering
- **Memory** — heap usage, garbage collection
- **Network** — all HTTP requests (see Supabase calls in real time)
- **Logging** — structured console logs

## Debugging in VS Code

1. Open the **mobile/** folder in VS Code
2. Press **F5** (or Run → Start Debugging)
3. Select a device from the dropdown
4. The app launches in debug mode. Set breakpoints by clicking the gutter.

### Useful debug operations:

- Step over: **F10**
- Step into: **F11**
- Continue to next breakpoint: **F5**
- Evaluate expression: hover over a variable, or open Debug Console and type

## Printing debug output

```
// Simple print (appears in VS Code debug console and terminal)
print('User profile: $profile');

// Better: use debugPrint for long strings (avoids truncation)
debugPrint('Full data: ${jsonEncode(data)}');
```

---

## §24 Writing and running tests

Flutter has three test levels. All tests go in the **test/** directory.

## Unit tests — test Dart logic in isolation

```
// test/core/utils/formatters_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:utamacs_portal/core/utils/formatters.dart';

void main() {
  group('formatters', () {
    test('formats currency correctly', () {
      expect(formatCurrency(1500), '₹1,500.00');
    });

    test('returns Resident for null name', () {
      expect(displayName(null), 'Resident');
    });
  });
}
```

Run unit tests:

```
flutter test test/core/
```

## Widget tests — test a single widget

```
// test/shared/widgets/status_badge_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:utamacs_portal/shared/widgets/status_badge.dart';

void main() {
  testWidgets('StatusBadge shows correct text', (WidgetTester tester) async {
    await tester.pumpWidget(
      const MaterialApp(home: Scaffold(body: StatusBadge(status:
        'approved'))),
    );
    expect(find.text('approved'), findsOneWidget);
  });
}
```

## Integration tests — test real app flows

Integration tests run on a real device or emulator. They test full user journeys.

```
// integration_test/auth_test.dart
import 'package:flutter_test/flutter_test.dart';
```

```
import 'package:integration_test/integration_test.dart';
import 'package:utamacs_portal/main.dart' as app;

void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets('Login screen shows email field', (WidgetTester tester) async {
    {
      app.main();
      await tester.pumpAndSettle();

      expect(find.text('Sign in with your email'), findsOneWidget);
      expect(find.byType(TextField), findsOneWidget);
    });
  }
}
```

Run integration tests:

```
flutter test integration_test/ -d macos
```

Run all tests

```
flutter test
```

---

## §25 Common errors and how to fix them

Could not find package "X" after adding a dependency

```
flutter pub get
```

.g.dart file is missing or out of date

```
dart run build_runner build --delete-conflicting-outputs
```

iOS build fails with CocoaPods could not find compatible versions

```
cd ios
pod install --repo-update
cd ..
flutter run
```

## iOS build fails with **Signing for "Runner" requires a development team**

1. Open **ios/Runner.xcworkspace** in Xcode
2. Select Runner target → Signing & Capabilities
3. Select your Apple Developer Team
4. Ensure "Automatically manage signing" is checked

## Android build fails with **SDK location not found**

```
# Check ANDROID_HOME is set correctly
echo $ANDROID_HOME
# Should be something like /Users/yourname/Library/Android/sdk

# If not set, add to ~/.zshrc:
export ANDROID_HOME=$HOME/Library/Android/sdk
source ~/.zshrc
```

## **Null check operator used on a null value** crash at runtime

This happens when you use **!** on a null value. Search for **!** in the crashing file and add null checks:

```
// Before (crashes if profile is null)
print(profile!.fullName);

// After (safe)
print(profile?.fullName ?? 'Unknown');
```

## **setState() called after dispose()**

This happens when an async operation completes after the widget has been removed. Fix with a mounted check:

```
if (!mounted) return; // add this before setState
setState(() => _loading = false);
```

## **RenderFlex overflowed** (yellow/black stripes on screen)

A **Row** or **Column** child is too wide/tall. Wrap the overflowing child in **Expanded** or **Flexible**:

```
// Before
Row(children: [Text('Very long text that overflows'),
Icon(Icons.arrow_right)])

// After
```

```
Row(children: [Expanded(child: Text('Very long text')),  
Icon(Icons.arrow_right)])
```

App not connecting to Supabase (`null` values everywhere)

1. Check `.env` exists at `mobile/.env`
2. Check `SUPABASE_URL` and `SUPABASE_ANON_KEY` are correct
3. Check `pubspec.yaml` has `.env` listed under assets
4. Check `await dotenv.load()` is called before `Supabase.initialize()`

`flutter doctor` shows red X for Android licences

```
flutter doctor --android-licenses  
# Type 'y' for each licence
```

---

## PART VI — BUILDING FEATURES

---

### §26 Adding a new screen (step by step)

Example: adding a Marketplace screen.

#### Step 1 — Create the repository file

```
// lib/features/marketplace/data/marketplace_repository.dart  
  
import 'package:riverpod_annotation/riverpod_annotation.dart';  
import 'package:supabase_flutter/supabase_flutter.dart';  
import '../core/constants/supabase.dart' as env;  
  
part 'marketplace_repository.g.dart';  
  
class MarketplaceListing {  
  final String id;  
  final String title;  
  final double price;  
  final String status;  
  
  const MarketplaceListing({  
    required this.id,  
    required this.title,  
    required this.price,  
    required this.status,  
  });  
  
  factory MarketplaceListing.fromJson(Map<String, dynamic> j) =>
```

```

MarketplaceListing(
  id: j['id'] as String,
  title: j['title'] as String,
  price: (j['price'] as num).toDouble(),
  status: j['status'] as String? ?? 'active',
);
}

@riverpod
MarketplaceRepository marketplaceRepository(MarketplaceRepositoryRef ref)
=>
  MarketplaceRepository();

class MarketplaceRepository {
  final _client = Supabase.instance.client;

  Future<List<MarketplaceListing>> fetchListings() async {
    final data = await _client
      .from('marketplace_listings')
      .select()
      .eq('society_id', env.societyId)
      .eq('status', 'active')
      .order('created_at', ascending: false)
      .limit(50);
    return (data as List).map((e) =>
MarketplaceListing.fromJson(e)).toList();
  }
}

@riverpod
Future<List<MarketplaceListing>> marketplaceListings(MarketplaceListingsRef
ref) =>
  ref.watch(marketplaceRepositoryProvider).fetchListings();

```

## Step 2 — Run code generation

```
dart run build_runner build --delete-conflicting-outputs
```

## Step 3 — Create the screen

```

// lib/features/marketplace/presentation/screens/marketplace_screen.dart

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../core/theme/app_theme.dart';
import '../data/marketplace_repository.dart';
import '../shared/widgets/empty_state.dart';

class MarketplaceScreen extends ConsumerWidget {

```

```

const MarketplaceScreen({super.key});

@override
Widget build(BuildContext context, WidgetRef ref) {
  final listingsAsync = ref.watch(marketplaceListingsProvider);

  return Scaffold(
    backgroundColor: kBgWarm,
    appBar: AppBar(
      title: const Text('Marketplace'),
      backgroundColor: Colors.white,
      surfaceTintColor: Colors.white,
    ),
    body: listingsAsync.when(
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (e, _) => Center(child: Text('Error: $e')),
      data: (listings) {
        if (listings.isEmpty) {
          return const EmptyState(
            icon: Icons.store_outlined,
            title: 'No listings yet',
            description: 'Items for sale will appear here.',
          );
        }
        return RefreshIndicator(
          onRefresh: () async =>
            ref.invalidate(marketplaceListingsProvider),
          child: ListView.builder(
            padding: const EdgeInsets.all(16),
            itemCount: listings.length,
            itemBuilder: (_, i) {
              final item = listings[i];
              return Card(
                margin: const EdgeInsets.only(bottom: 12),
                child: ListTile(
                  title: Text(item.title),
                  trailing: Text('₹${item.price.toStringAsFixed(0)}'),
                ),
              );
            },
          ),
        );
      },
    ),
  );
}

```

#### Step 4 — Register the route in `app.dart`

Inside the `ShellRoute.routes` list:

```
GoRoute(
  path: '/marketplace',
  builder: (ctx, st) => const MarketplaceScreen(),
),
```

### Step 5 — Add to the Services grid in `services_screen.dart`

Add to the appropriate section:

```
_ServiceItem(
  label: 'Marketplace',
  icon: Icons.store_outlined,
  bg: Color(0xFFFFF3CD),
  fg: Color(0xFFD97706),
  route: '/marketplace',
),
```

### Step 6 — Test it

```
flutter run -d macos
# Navigate to Services → tap Marketplace
```

## §27 Adding a new Supabase query

Pattern for queries that need parameters (e.g., get events for a specific month):

```
// In the repository class
Future<List<Event>> fetchEventsForMonth(int year, int month) async {
  final start = DateTime(year, month, 1).toIso8601String();
  final end = DateTime(year, month + 1, 1).toIso8601String();

  final data = await _client
    .from('events')
    .select()
    .eq('society_id', env.societyId)
    .gte('event_date', start) // gte = greater than or equal
    .lt('event_date', end)   // lt = less than
    .order('event_date');
  return (data as List).map((e) => Event.fromJson(e)).toList();
}

// Define a family provider (takes parameters)
@riverpod
Future<List<Event>> eventsForMonth(EventsForMonthRef ref, int year, int
```

```
month) =>
    ref.watch(eventRepositoryProvider).fetchEventsForMonth(year, month);
```

Using a family provider in a widget:

```
final eventsAsync = ref.watch(eventsForMonthProvider(2026, 5));
```

## §28 Adding a form with validation

```
class SubmitComplaintScreen extends ConsumerStatefulWidget {
  const SubmitComplaintScreen({super.key});

  @override
  ConsumerState<SubmitComplaintScreen> createState() =>
    _SubmitComplaintScreenState();
}

class _SubmitComplaintScreenState
  extends ConsumerState<SubmitComplaintScreen> {
  final _formKey = GlobalKey<FormState>();
  final _titleCtrl = TextEditingController();
  final _descCtrl = TextEditingController();
  String _category = 'maintenance';
  bool _loading = false;

  @override
  void dispose() {
    _titleCtrl.dispose();
    _descCtrl.dispose();
    super.dispose();
  }

  Future<void> _submit() async {
    if (!_formKey.currentState!.validate()) return; // runs all validators
    setState(() => _loading = true);
    try {
      await ref.read(complaintRepositoryProvider).createComplaint(
        title: _titleCtrl.text.trim(),
        description: _descCtrl.text.trim(),
        category: _category,
      );
      if (!mounted) return;
      context.pop(); // go back
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Complaint submitted')),
      );
    } catch (e) {
      if (!mounted) return;
    }
  }
}
```

```

        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Failed: $e'), backgroundColor: kRed600),
        );
      } finally {
        if (mounted) setState(() => _loading = false);
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Submit Complaint')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(20),
          children: [
            TextFormField(
              controller: _titleCtrl,
              decoration: const InputDecoration(labelText: 'Title'),
              validator: (v) {
                if (v == null || v.trim().isEmpty) return 'Title is
required';

                if (v.trim().length < 5) return 'Title is too short';
                return null; // null means valid
              },
            ),
            const SizedBox(height: 16),
            TextFormField(
              controller: _descCtrl,
              decoration: const InputDecoration(labelText: 'Description'),
              maxLines: 4,
              validator: (v) {
                if (v == null || v.trim().isEmpty) return 'Description is
required';

                return null;
              },
            ),
            const SizedBox(height: 24),
            ElevatedButton(
              onPressed: _loading ? null : _submit,
              child: _loading
                ? const SizedBox(
                    height: 20, width: 20,
                    child: CircularProgressIndicator(color: Colors.white,
strokeWidth: 2),
                  )
                : const Text('Submit'),
            ),
          ],
        ),
      ),
    );
  }

```

```
}  
}
```

## §29 Adding file/image upload from mobile

The portal stores files in the private GitHub docs repository via `commitDocument()` in the web API. Mobile uploads should NOT call GitHub directly — instead they POST to the portal's `/api/v1/` endpoints which handle the GitHub commit server-side.

### Step 1 — Add `image_picker` to `pubspec.yaml`

```
dependencies:  
  image_picker: ^1.1.2
```

Then `flutter pub get`.

### Step 2 — Add platform permissions

iOS — add to `ios/Runner/Info.plist`:

```
<key>NSCameraUsageDescription</key>  
<string>Take photos for complaint attachments.</string>  
<key>NSPhotoLibraryUsageDescription</key>  
<string>Select photos from library for complaint attachments.</string>
```

Android — add to `android/app/src/main/AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.CAMERA" />
```

### Step 3 — Pick an image

```
import 'package:image_picker/image_picker.dart';  
  
final _picker = ImagePicker();  
  
Future<void> _pickImage() async {  
  final file = await _picker.pickImage(  
    source: ImageSource.gallery, // or ImageSource.camera  
    maxWidth: 1920,  
    maxHeight: 1920,  
    imageQuality: 85, // 0-100, reduces file size  
  );  
  if (file == null) return; // user cancelled
```

```

final bytes = await file.readAsBytes();
final mimeType = file.mimeType ?? 'image/jpeg';

// POST to portal API
await _uploadToPortal(bytes, mimeType, file.name);
}

```

## Step 4 — Upload to portal API

```

import 'package:http/http.dart' as http;
import 'dart:convert';

Future<void> _uploadToPortal(
  Uint8List bytes,
  String mimeType,
  String filename,
) async {
  final session = Supabase.instance.client.auth.currentSession;
  if (session == null) throw Exception('Not authenticated');

  final uri =
Uri.parse('${env.portalApiBase}/api/v1/complaints/$complaintId/attachments'
);
  final request = http.MultipartRequest('POST', uri)
    ..headers['Authorization'] = 'Bearer ${session.accessToken}'
    ..files.add(http.MultipartFile.fromBytes(
      'file',
      bytes,
      filename: filename,
      contentType: MediaType.parse(mimeType),
    ));

  final response = await request.send();
  if (response.statusCode != 200) {
    throw Exception('Upload failed: ${response.statusCode}');
  }
}

```

Note: `portalApiBase` needs to be added to `.env` (e.g., `PORTAL_API_BASE=https://portal.utamacs.org`).

## §30 Push notifications

Push notifications require Firebase Cloud Messaging (FCM). This is not yet implemented in the current build. Here is the full setup guide for when you are ready.

### Step 1 — Firebase project setup

1. Go to <https://console.firebase.google.com>

2. Create a project "utamacs-portal" (or add to existing)
3. Add Android app: package name `org.utamacs.utamacs_portal`
4. Add iOS app: bundle ID `org.utamacs.utamacsPortal`
5. Download `google-services.json` → place in `android/app/`
6. Download `GoogleService-Info.plist` → place in `ios/Runner/`

## Step 2 — Add dependencies

```
dependencies:  
  firebase_core: ^3.0.0  
  firebase_messaging: ^15.0.0  
  flutter_local_notifications: ^18.0.1 # already in pubspec
```

## Step 3 — Initialise Firebase in main.dart

```
import 'package:firebase_core/firebase_core.dart';  
import 'firebase_options.dart'; // generated by FlutterFire CLI  
  
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp(options:  
DefaultFirebaseOptions.currentPlatform);  
  await dotenv.load();  
  await Supabase.initialize(...);  
  runApp(const ProviderScope(child: UtamacsApp()));  
}
```

## Step 4 — Request permission and get FCM token

```
final messaging = FirebaseMessaging.instance;  
  
// Request permission (iOS requires explicit permission)  
await messaging.requestPermission(alert: true, badge: true, sound: true);  
  
// Get FCM token and store it in Supabase  
final token = await messaging.getToken();  
if (token != null) {  
  await Supabase.instance.client  
    .from('notification_subscriptions')  
    .upsert({'user_id': uid, 'fcm_token': token, 'platform':  
Platform.isIOS ? 'ios' : 'android'});  
}  
  
// Refresh token when it changes  
messaging.onTokenRefresh.listen((token) {
```

```
// Update in Supabase
});
```

## Step 5 — Handle incoming notifications

```
// Foreground notifications
FirebaseMessaging.onMessage.listen((message) {
  // Show a local notification
  flutterLocalNotificationsPlugin.show(
    0,
    message.notification?.title,
    message.notification?.body,
    const NotificationDetails(...),
  );
});

// When user taps a notification that opens the app
FirebaseMessaging.onMessageOpenedApp.listen((message) {
  // Navigate based on message.data
  final route = message.data['route'];
  if (route != null) context.go(route);
});
```

---

# PART VII — DEPLOYING TO PRODUCTION

---

## §31 Platform preparation — Android signing

**Why:** Google Play requires all APKs/AABs to be signed with a private key. Debug builds are signed automatically with a debug key. Release builds need your own key.

### Create a keystore (one-time setup)

```
# Run from the mobile/ directory
# Replace values with your real information
keytool -genkey -v \
  -keystore android/app/utamacs-release-key.jks \
  -alias utamacs \
  -keyalg RSA \
  -keysize 2048 \
  -validity 10000 \
  -dname "CN=UTAMACS, OU=Mobile, O=UTA MACS, L=Hyderabad, S=Telangana, C=IN"

# You will be prompted to set a keystore password and key password
# IMPORTANT: Remember these passwords and store the .jks file safely
```

```
# If you lose this file or the password, you cannot update your app on Play Store
```

**NEVER commit `utamacs-release-key.jks` to git.** Add it to `.gitignore`:

```
android/app/*.jks
android/key.properties
```

## Create key.properties

Create `android/key.properties` (also gitignored):

```
storePassword=your_keystore_password
keyPassword=your_key_password
keyAlias=utamacs
storeFile=utamacs-release-key.jks
```

## Wire up signing in build.gradle.kts

Edit `android/app/build.gradle.kts`:

```
// Add at the top, before android {}
import java.util.Properties
import java.io.FileInputStream

val keystorePropertiesFile = rootProject.file("../key.properties")
val keystoreProperties = Properties()
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(FileInputStream(keystorePropertiesFile))
}

android {
    // ... existing config ...

    signingConfigs {
        create("release") {
            keyAlias = keystoreProperties["keyAlias"] as String
            keyPassword = keystoreProperties["keyPassword"] as String
            storeFile = file(keystoreProperties["storeFile"] as String)
            storePassword = keystoreProperties["storePassword"] as String
        }
    }

    buildTypes {
        getByName("release") {
            signingConfig = signingConfigs.getByName("release")
            isMinifyEnabled = true
        }
    }
}
```

```
        isShrinkResources = true
        proguardFiles(
            getDefaultProguardFile("proguard-android-optimize.txt"),
            "proguard-rules.pro"
        )
    }
}
```

## Test the signed build

```
flutter build apk --release
# Output: build/app/outputs/flutter-apk/app-release.apk
# Install on a physical device to verify:
adb install build/app/outputs/flutter-apk/app-release.apk
```

---

## §32 Platform preparation — iOS signing and certificates

**Why:** Apple requires all apps (even on TestFlight) to be signed with a certificate from your Apple Developer account.

### Prerequisite: Apple Developer Account

1. Go to <https://developer.apple.com/programs/enroll/>
2. Enrol as Individual or Organisation
3. Pay \$99 USD / year
4. Wait for approval (usually instant for Individual, 2–7 days for Organisation)

### Set the Bundle ID in Xcode

1. Open `mobile/ios/Runner.xcworkspace` in Xcode (not `.xcodeproj` — open the workspace)
2. In the Project Navigator (left panel), click "Runner"
3. Select the "Runner" target
4. Under "General" tab:
  - Display Name: `UTA MACS`
  - Bundle Identifier: `org.utamacs.utamacsPortal`
  - Version: `1.0.0`
  - Build: `1`

### Set up signing in Xcode

1. Select Runner target → Signing & Capabilities tab
2. Check "Automatically manage signing"
3. Under Team: select your Apple Developer account
4. Xcode will automatically create:
  - An App ID on Apple Developer portal

- A signing certificate
- A provisioning profile

If you see errors:

- "No account for team" → Add your Apple ID in Xcode → Settings → Accounts → + button
- "Failed to create provisioning profile" → You may need to register your device UDID in the Apple Developer portal

## Set minimum iOS version

In Xcode → Runner target → General → Deployment Info: set minimum to iOS 14.0 (or higher, but not lower — Flutter requires at least iOS 12, and most users are on 15+).

## Add required Info.plist entries

Edit `ios/Runner/Info.plist` (XML format). Add before the closing `</dict>`:

```
<!-- Camera – for QR scanning and complaint photos -->
<key>NSCameraUsageDescription</key>
<string>Camera is used to scan visitor QR codes and attach photos to
complaints.</string>

<!-- Photo library – for community posts and complaint attachments -->
<key>NSPhotoLibraryUsageDescription</key>
<string>Select photos from your library for posts and complaints.</string>

<!-- Photo library add – for saving QR codes to camera roll -->
<key>NSPhotoLibraryAddUsageDescription</key>
<string>Save visitor pass QR codes to your photo library.</string>
```

## §33 Building release binaries

Bump the version number first

In `pubspec.yaml`:

```
version: 1.0.0+1
#      ^ ^
#      | build number (integer, must increment with every store upload)
#      version string (shown to users)
```

For example, next release: `version: 1.0.1+2`

## Android — App Bundle (for Play Store)

```
flutter build appbundle --release
# Output: build/app/outputs/bundle/release/app-release.aab
```

Use **.aab** for Play Store (not **.apk**). Google Play uses the bundle to generate optimised APKs for each device type.

For direct APK distribution (not Play Store):

```
flutter build apk --release --split-per-abi
# Creates separate APKs per CPU architecture – smaller file sizes
# Outputs:
#   app-armeabi-v7a-release.apk    (32-bit ARM – older phones)
#   app-arm64-v8a-release.apk     (64-bit ARM – most modern phones)
#   app-x86_64-release.apk        (64-bit x86 – emulators and some tablets)
```

## iOS — Archive for App Store

```
flutter build ios --release
```

After this completes, upload to App Store via Xcode Organiser:

1. In Xcode: Product → Archive
2. Wait for the archive to build (5–15 minutes)
3. Xcode Organiser opens automatically showing the new archive
4. Click "Distribute App" → App Store Connect → Upload
5. Follow the prompts

## Build numbers

The build number (+1, +2) must be a positive integer that increments with every upload to the store. It does not need to match the version string. The easiest approach:

```
# Pass version and build number on the command line
flutter build appbundle --release --build-name=1.0.1 --build-number=5
flutter build ios --release --build-name=1.0.1 --build-number=5
```

---

## §34 Deploying to Google Play Store

### Step 1 — Create a Play Console account

1. Go to <https://play.google.com/console>
2. Sign in with a Google account

3. Pay \$25 USD (one-time, not annual)
4. Complete the account setup (fill in developer name, contact email)

## Step 2 — Create the app

1. Click "Create app"
2. App name: "UTA MACS"
3. Default language: English (India)
4. App or Game: App
5. Free or Paid: Free
6. Accept Developer Program Policies and Play App Signing agreements
7. Click "Create app"

## Step 3 — Set up Play App Signing (important)

Google Play manages the final signing key on its servers. You upload your app bundle signed with your upload key (from §31), and Google re-signs it with the Play App Signing key before distribution.

1. In your app → Setup → App signing
2. Select "Use Google-managed key"
3. Follow the prompts

This means: if you ever lose your upload keystore, Google can help you recover. Do not lose your keystore anyway.

## Step 4 — Fill in the store listing

Go to Store presence → Main store listing:

### Short description (80 characters):

Resident portal for UTA MACS — notices, visitors, complaints & more.

**Full description (4000 characters):** Write a description covering:

- What the app does (resident portal for UTAMACS society)
- Key features (visitor management, notices, complaints, dues, events)
- Who it is for (UTAMACS residents in Kondakal, Shankarpalle)

### Screenshots (required):

- Phone screenshots: at least 2, up to 8. Use the emulator to take them.
  - Simulator: Press Cmd+S (macOS Simulator) or use `adb exec-out screencap -p > screen.png` on Android
- Feature graphic: 1024×500 JPEG/PNG — a banner image for the store listing

**App icon:** 512×512 PNG, no alpha channel.

## Step 5 — Content rating questionnaire

Dashboard → Policy → App content → Content ratings:

- Fill in the questionnaire honestly
- UTAMACS is a utility app with no violence, mature content, gambling, etc.
- You will receive a rating of "Everyone" or similar

## Step 6 — Target audience

Policy → App content → Target audience:

- Set target age to "18 and over" (society management app)
- No ads targeting children

## Step 7 — Privacy policy

You need a privacy policy URL. Create one at <https://utamacs.org/privacy> that covers:

- What data is collected (email, name, unit number, visitor information)
- How it is used (society management)
- DPDPA 2023 compliance
- Contact information for data requests

Enter the URL in: Policy → App content → Privacy policy.

## Step 8 — Upload the first build

Go to Release → Testing → Internal testing:

1. Click "Create new release"
2. Click "Upload" and select your **.aab** file
3. Add release notes (e.g., "Initial release for UTAMACS residents")
4. Click "Save" then "Review release"
5. Click "Start rollout to Internal testing"

Add yourself and trusted testers as internal testers:

- Testing → Internal testing → Testers tab
- Add Google accounts of testers

## Step 9 — Promote to production

After testing:

1. Release → Production → Create new release
2. Either upload a new build or promote the internal testing build
3. Set rollout percentage (start at 20% to catch issues before full rollout)
4. Click "Review release" → "Start rollout to Production"

**Google review typically takes 1–3 days for the first submission.**

---

## §35 Deploying to Apple App Store

### Step 1 — App Store Connect setup

1. Go to <https://appstoreconnect.apple.com>
2. Sign in with your Apple Developer account
3. Click "My Apps" → "+" → "New App"
4. Fill in:
  - Platform: iOS
  - Name: "UTA MACS"
  - Primary language: English (India) or English
  - Bundle ID: `org.utamacs.utamacsPortal` (must match exactly)
  - SKU: `utamacs-resident-portal` (internal identifier, not shown to users)
  - User access: Full Access

## Step 2 — Prepare app metadata

### App Information:

- Category: Utilities (or Productivity)
- Content Rights: UTAMACS owns all content

### Pricing:

- Price: Free

**App Privacy:** Go to App Privacy → Privacy Nutrition Label. You must declare:

- Data types collected: email address (required), name (optional), device identifier (analytics)
- Purpose: App functionality
- Whether data is linked to identity: Yes (email/name), No (crash data)

## Step 3 — Prepare screenshots

Required sizes:

- **6.7" (iPhone 15 Plus):** 1290×2796 pixels — run on iPhone 15 Plus simulator
- **5.5" (iPhone 8 Plus):** 1242×2208 pixels — run on iPhone 8 Plus simulator (legacy but required)

Taking screenshots:

1. Run `flutter run -d "iPhone 15 Plus"` in Simulator
2. Navigate to the screen
3. Cmd+S to save screenshot
4. Repeat for each screen you want to showcase

Suggested screenshots:

1. Login screen (shows brand identity)
2. Dashboard (shows quick services grid)
3. Visitor pass with QR code
4. Notices list
5. Profile screen

**App Preview video** (optional but recommended): 30-second screen recording showing key features.

## Step 4 — Fill in the store listing

**Name:** UTA MACS (30 characters max)

**Subtitle:** Resident Portal (30 characters max)

**Keywords:** society, resident, apartment, maintenance, visitors, notices (100 characters total, comma-separated)

**Description (4000 characters):** Describe the app for residents. Cover: notices and circulars, visitor management, complaints, dues and finance, events, community, facilities.

**Support URL:** <https://utamacs.org/contact>

**Marketing URL:** <https://utamacs.org> (optional)

**What's New in This Version:** "Initial release of the UTA MACS Resident Portal for iOS."

## Step 5 — Submit for review

1. Build → select the uploaded build (from Xcode Organiser upload in §33)
2. Fill in all required fields (screenshots, description, ratings)
3. Answer the Export Compliance question: No encryption other than HTTPS (select "Yes, it uses encryption" if you consider TLS)
4. Click "Add for Review" → "Submit to App Review"

**Apple review takes 1–3 days** for the first submission. Subsequent updates are usually reviewed within 24 hours.

## Common App Store rejection reasons (and how to avoid them)

| Rejection reason                                     | How to avoid                                                             |
|------------------------------------------------------|--------------------------------------------------------------------------|
| Missing privacy policy                               | Always provide a real privacy policy URL                                 |
| App crashes on reviewer's device                     | Test on multiple devices before submitting                               |
| Login required but no demo account                   | Add a note in "App Review Information" with a demo email and explain OTP |
| Guideline 4.0 Design: UI is not up to Apple standard | Ensure standard iOS elements, proper safe area insets                    |
| Guideline 2.1: App Completeness                      | Remove placeholder screens, all links must work                          |

**For the OTP login:** In the App Review Information section, add a note:

"This app uses passwordless email authentication. To test, use demo@utamacs.org. The 6-digit OTP will be sent to that email. Contact testaccess@utamacs.org to receive the code during review."

## Step 6 — TestFlight beta testing

Before submitting to the App Store, use TestFlight:

1. Upload a build via Xcode Organisier
2. In App Store Connect → TestFlight → builds will appear (wait for processing, ~30 minutes)
3. Internal testers: up to 100 Apple Developer accounts — no review needed
4. External testers: up to 10,000 testers by email — requires a brief beta review (1–2 days)

Invite testers:

- TestFlight → Internal Testing → + button → add Apple ID emails
- Testers install the TestFlight app from App Store, then accept your invitation

---

## §36 CI/CD pipeline with GitHub Actions

Automate builds and deployments so you do not have to run them manually.

Basic CI (test on every PR)

Create `.github/workflows/mobile-ci.yml`:

```
name: Mobile CI

on:
  push:
    branches: [main]
    paths:
      - 'mobile/**'
  pull_request:
    branches: [main]
    paths:
      - 'mobile/**'

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: subosito/flutter-action@v2
        with:
          flutter-version: '3.27.0' # pin to your current version
          channel: 'stable'

      - name: Install dependencies
        working-directory: mobile
        run: flutter pub get

      - name: Run code generation
        working-directory: mobile
        run: dart run build_runner build --delete-conflicting-outputs
```

- name: Analyse code  
working-directory: mobile  
run: flutter analyze
- name: Run tests  
working-directory: mobile  
run: flutter test

## Android release build (triggered on version tags)

```

name: Android Release

on:
  push:
    tags:
      - 'v*.*.*' # triggers on tags like v1.0.0

jobs:
  build-android:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: subosito/flutter-action@v2
        with:
          flutter-version: '3.27.0'
          channel: 'stable'

      - name: Create .env file
        working-directory: mobile
        run: |
          echo "SUPABASE_URL=${{ secrets.SUPABASE_URL }}" > .env
          echo "SUPABASE_ANON_KEY=${{ secrets.SUPABASE_ANON_KEY }}" >> .env
          echo "SOCIETY_ID=${{ secrets.SOCIETY_ID }}" >> .env

      - name: Create key.properties
        working-directory: mobile
        run: |
          echo "storePassword=${{ secrets.KEYSTORE_PASSWORD }}" >
android/key.properties
          echo "keyPassword=${{ secrets.KEY_PASSWORD }}" >>
android/key.properties
          echo "keyAlias=utamacs" >> android/key.properties
          echo "storeFile=utamacs-release-key.jks" >>
android/key.properties

      - name: Decode keystore
        working-directory: mobile/android/app
        run: |
          echo "${{ secrets.KEYSTORE_BASE64 }}" | base64 --decode >
utamacs-release-key.jks

```

```

- name: Build App Bundle
  working-directory: mobile
  run: flutter build appbundle --release

- name: Upload to Play Store (internal track)
  uses: r0adkll/upload-google-play@v1
  with:
    serviceAccountJsonPlainText: ${
secrets.PLAY_SERVICE_ACCOUNT_JSON }}
    packageName: org.utamacs.utamacs_portal
    releaseFiles: mobile/build/app/outputs/bundle/release/app-
release.aab
    track: internal
    status: completed

```

## GitHub Secrets to configure

In GitHub repo → Settings → Secrets and variables → Actions → New repository secret:

| Secret name                            | Value                                                                                                    |
|----------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>SUPABASE_URL</code>              | Your Supabase project URL                                                                                |
| <code>SUPABASE_ANON_KEY</code>         | Your Supabase anon key                                                                                   |
| <code>SOCIETY_ID</code>                | The UTAMACS society UUID                                                                                 |
| <code>KEYSTORE_PASSWORD</code>         | The keystore password from §31                                                                           |
| <code>KEY_PASSWORD</code>              | The key password from §31                                                                                |
| <code>KEYSTORE_BASE64</code>           | The <code>.jks</code> file encoded as base64: <code>base64 -i android/app/utamacs-release-key.jks</code> |
| <code>PLAY_SERVICE_ACCOUNT_JSON</code> | JSON key for a Google Play service account (see Play Console → Setup → API access)                       |

## PART VIII — MAINTENANCE

### §37 Upgrading packages

Check what is outdated

```

cd mobile
flutter pub outdated

```

This shows current version, latest resolvable version, and latest available version for each package.

## Upgrade a specific package

```
flutter pub upgrade supabase_flutter
```

## Upgrade all packages to latest compatible versions

```
flutter pub upgrade
```

## Upgrade with version constraint bumps (breaking changes possible)

```
flutter pub upgrade --major-versions
```

**Always test the app after any dependency upgrade.** Breaking changes in packages (especially `supabase_flutter`, `go_router`, `riverpod`) can require code changes.

## Upgrading Flutter itself

```
# Check current version
flutter --version

# Upgrade to latest stable
flutter upgrade

# Check available channels
flutter channel
# stable = production-ready
# beta = upcoming features, generally stable
# master = bleeding edge, may have bugs
```

After upgrading Flutter, run `flutter pub get` and rebuild.

## After any upgrade — checklist

```
cd mobile
flutter pub get
dart run build_runner build --delete-conflicting-outputs
flutter analyze
flutter test
flutter run -d macos # quick smoke test
```

---

## §38 Commit history — what landed and when

## Commit 1 — feat: Flutter mobile app scaffold — Auth, Dashboard, Notices, Visitors (c506466)

Full Flutter project: `main.dart`, `app.dart` (GoRouter + 5-tab shell), `app_theme.dart`, `AuthRepository`, `AuthNotifier`, `LoginScreen` (email OTP), `DashboardScreen` (header + services grid), `NoticesScreen` + `NoticeDetailScreen`, `VisitorsScreen` + `PreApproveScreen` + `VisitorPassScreen` (QR code), `Profile` model, `AppCard`, `EmptyState`, `StatusBadge`, `.env.example`.

## Commit 2 — fix: mobile auth, visitor pass schema fixes, and profiles RLS recursion (89a55b9)

Switched auth to email OTP. Added macOS target. Switched to `google_fonts`. Fixed visitor pass insert: `otp_code` NOT NULL (generates 6-digit OTP), `expected_date` format (YYYY-MM-DD), `expires_at` NOT NULL (defaults to +24h), `host_unit_id` lookup from profiles before insert. Fixed `vehicle_number` field. Fixed profiles RLS recursion.

## Commit 3 — feat: mobile app UI redesign — warm design language, 5-tab nav, service grid (79b6cbd)

Scaffold background → `kBgWarm` (#F5F0EB). Dashboard redesigned with expandable quick-services grid (8 primary + 4 expandable). Services tab added as "All Services" screen with 4 sections. Material 3 `NavigationBar` refined.

## Commit 4 — feat: port all 28 portal modules to Flutter mobile app (283a503)

24 remaining modules added: Complaints, Finance, Events, Polls, Community, Documents, Facilities, Parking, Maids, Members, Notifications, Gallery, Water Tankers, Vendors, Feedback, Snags, Security Patrol, Policies, Register, AGM, Tenant KYC, HOTO, Letters, Analytics, Staff. All routes registered.

---

## §39 Known issues and bugs fixed

### RLS recursion on profiles table

**Symptom:** Profile fetch hangs or returns error. **Root cause:** RLS policy on `profiles` table was doing a self-referential subquery. **Fix:** Profile query uses direct `.eq('id', uid)` primary key lookup. RLS policy corrected.

### Visitor pass insert — three schema mismatches

1. `otp_code` is NOT NULL — fix: generate 6-digit OTP on client before insert.
2. `expected_date` is `date` type, not `timestampz` — fix: format as YYYY-MM-DD.
3. `expires_at` is NOT NULL — fix: default to `expected_date + Duration(hours: 24)`.

### `host_unit_id` required but not fetched

`visitor_pre_approvals` requires both `host_user_id` (user UUID) and `host_unit_id` (unit UUID). Fix: look up `unit_id` from `profiles` table before inserting the approval.

## §40 Next steps and roadmap

### Priority 1 — Required before any internal user testing

- ☐ **App icons** — Replace Flutter default icon with UTAMACS logo. Use `flutter_launcher_icons` package: add `flutter_launcher_icons: ^0.14.1` to dev\_dependencies, create `flutter_launcher_icons.yaml`, run `dart run flutter_launcher_icons`. Provide a 1024×1024 PNG master icon.
- ☐ **Splash screen** — Replace white Flutter splash with branded. Use `flutter_native_splash` package. Provide a 1:1 logo on the UTAMACS primary colour background.
- ☐ **Android permissions** — Add to `android/app/src/main/AndroidManifest.xml`: `INTERNET, CAMERA, VIBRATE, RECEIVE_BOOT_COMPLETED`.
- ☐ **iOS permissions** — Add to `ios/Runner/Info.plist`: `NSCameraUsageDescription, NSPhotoLibraryUsageDescription`.
- ☐ **iOS Bundle ID** — Set to `org.utamacs.utamacsPortal` in Xcode as described in §32.
- ☐ **Android signing** — Complete the keystore setup in §31 before any release build.

### Priority 2 — Core functionality gaps

- ☐ **Write operations for the 24 module screens** — Complaints (submit form done), but Finance (pay dues), Events (RSVP), Polls (vote), Community (create post), Facilities (book slot), Water Tankers (book tanker), Feedback (submit), Snags (report — screen exists) all need their write forms wired up.
- ☐ **QR scanner for security guards** — `mobile_scanner` is a dependency. Build a scanner screen: `CameraPreview` → decode QR JSON → call `/api/v1/visitors/verify` → show admit/deny UI.
- ☐ **File upload** — Implement §29 for complaint attachments, community post images, and snag photos.
- ☐ **Pagination** — All list screens use `limit(20–50)`. Add infinite scroll using `ScrollController.atEdge` to trigger the next page.
- ☐ **Pull-to-refresh** — `RefreshIndicator` wrapping every list screen.
- ☐ **Offline/error states** — Every `AsyncValue.error` branch needs a retry button.

### Priority 3 — Polish

- ☐ **Shimmer loading** — Replace `CircularProgressIndicator` with shimmer card placeholders on list screens.
- ☐ **Role-based visibility** — Hide management modules (Analytics, HOTO, Staff) from `member` role users in the Services grid.
- ☐ **Guard home screen** — Security guards should land on a guard-specific home (visitor queue + QR scanner), not the resident dashboard.
- ☐ **Biometric login** — Add `local_auth` package. After OTP login, optionally enable Face ID / fingerprint for subsequent logins.
- ☐ **Dark mode** — Add `darkTheme: _buildDarkTheme()` to `MaterialApp.router`.
- ☐ **Telugu localisation** — Add `flutter_localizations` + ARB files for `te` (Telugu) locale.

### Priority 4 — Production release

- ☐ **Privacy policy** — Create <https://utamacs.org/privacy> covering DPDPA 2023.
- ☐ **App Store screenshots** — Take screenshots on iPhone 15 Plus (6.7") and iPhone 8 Plus (5.5") simulators.
- ☐ **Play Store listing** — Feature graphic, store description, screenshots.
- ☐ **Apple Developer Program** — Enrol and pay \$99/year.
- ☐ **TestFlight beta** — Upload build, invite 10 residents to test.
- ☐ **Play internal testing** — Upload AAB, invite same residents.
- ☐ **GitHub Actions CI** — Set up the workflow from §36.

## §41 Reference links

### Official documentation

| Resource                  | URL                                                                                                                         |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Flutter docs (start here) | <a href="https://docs.flutter.dev">https://docs.flutter.dev</a>                                                             |
| Flutter install (macOS)   | <a href="https://docs.flutter.dev/get-started/install/macos">https://docs.flutter.dev/get-started/install/macos</a>         |
| Dart language tour        | <a href="https://dart.dev/language">https://dart.dev/language</a>                                                           |
| Dart cheatsheet           | <a href="https://dart.dev/codelabs/dart-cheatsheet">https://dart.dev/codelabs/dart-cheatsheet</a>                           |
| Flutter widget catalogue  | <a href="https://docs.flutter.dev/ui/widgets">https://docs.flutter.dev/ui/widgets</a>                                       |
| Material 3 components     | <a href="https://m3.material.io/components">https://m3.material.io/components</a>                                           |
| Riverpod documentation    | <a href="https://riverpod.dev/docs/introduction/getting_started">https://riverpod.dev/docs/introduction/getting_started</a> |
| go_router documentation   | <a href="https://pub.dev/packages/go_router">https://pub.dev/packages/go_router</a>                                         |
| Supabase Flutter docs     | <a href="https://supabase.com/docs/reference/dart">https://supabase.com/docs/reference/dart</a>                             |
| Flutter DevTools          | <a href="https://docs.flutter.dev/tools/devtools/overview">https://docs.flutter.dev/tools/devtools/overview</a>             |

### Learning resources (for Flutter/Dart newcomers)

| Resource                                                              | What it covers                                                                                          |
|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Flutter: Your first app codelab                                       | <a href="https://docs.flutter.dev/get-started/codelab">https://docs.flutter.dev/get-started/codelab</a> |
| Dart language tour                                                    | <a href="https://dart.dev/language">https://dart.dev/language</a> — complete reference                  |
| Flutter & Dart - The Complete Guide (Udemy, Maximilian Schwarzmüller) | Best paid course, covers everything in depth                                                            |
| The Net Ninja Flutter series (YouTube, free)                          | Good free beginner series                                                                               |
| Flutter Mapp (YouTube)                                                | Good Riverpod and advanced topics                                                                       |
| Official Flutter YouTube channel                                      | <a href="https://youtube.com/@flutterdev">https://youtube.com/@flutterdev</a>                           |
| Flutter Community on Reddit                                           | <a href="https://reddit.com/r/FlutterDev">https://reddit.com/r/FlutterDev</a>                           |

| Resource               | What it covers                                                        |
|------------------------|-----------------------------------------------------------------------|
| Flutter Discord server | <a href="https://discord.gg/rflutter">https://discord.gg/rflutter</a> |

## App store references

| Resource                 | URL                                                                                                                                                         |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Google Play Console      | <a href="https://play.google.com/console">https://play.google.com/console</a>                                                                               |
| Play Store policies      | <a href="https://support.google.com/googleplay/android-developer/answer/9858738">https://support.google.com/googleplay/android-developer/answer/9858738</a> |
| Apple App Store Connect  | <a href="https://appstoreconnect.apple.com">https://appstoreconnect.apple.com</a>                                                                           |
| App Review Guidelines    | <a href="https://developer.apple.com/app-store/review/guidelines">https://developer.apple.com/app-store/review/guidelines</a>                               |
| TestFlight documentation | <a href="https://developer.apple.com/testflight">https://developer.apple.com/testflight</a>                                                                 |
| Apple Developer Program  | <a href="https://developer.apple.com/programs/enroll">https://developer.apple.com/programs/enroll</a>                                                       |

## Useful tools

| Tool                                   | Purpose                                                                                       |
|----------------------------------------|-----------------------------------------------------------------------------------------------|
| <a href="#">pub.dev</a>                | Package registry — search Flutter packages                                                    |
| <a href="#">dartpad.dev</a>            | Online Dart/Flutter playground — test code snippets without setup                             |
| DartFrog                               | Backend framework for Dart if you ever want a Dart API server                                 |
| FlutterFire CLI                        | <code>dart pub global activate flutterfire_cli</code> — sets up Firebase in a Flutter project |
| <a href="#">flutter_launcher_icons</a> | Generate app icons for all platforms from a single source image                               |
| <a href="#">flutter_native_splash</a>  | Generate native splash screens                                                                |
| Codemagic                              | CI/CD service with good Flutter support                                                       |
| Fastlane                               | Automation tool for iOS/Android delivery                                                      |

## UTAMACS project references

| Resource                  | Location                             |
|---------------------------|--------------------------------------|
| Web portal source         | <a href="#">src/</a>                 |
| Supabase migrations       | <a href="#">supabase/migrations/</a> |
| Design tokens (Tailwind)  | <a href="#">tailwind.config.cjs</a>  |
| CLAUDE.md (project rules) | <a href="#">CLAUDE.md</a>            |

| Resource                  | Location                                        |
|---------------------------|-------------------------------------------------|
| Mobile stack decision doc | <a href="#">design/mobile-stack-decision.md</a> |
| This runbook              | <a href="#">design/MOBILE-RUNBOOK.md</a>        |

---

*This runbook is the authoritative, self-sufficient guide for the UTAMACS mobile app. Update it every time something significant changes — a new dependency, a deployment, a new screen pattern, a bug fix. The person who reads it next should not need to ask anyone questions.*